

VesicleDetection Handbook

Richie Dadhley

April 7, 2025

Contents

1	Introduction	2
2	Prerequisites	2
2.1	Zarr Data	2
2.2	Gunpowder	3
2.3	UNet	3
2.4	Hough transform	4
2.5	Napari	4
3	The Code	4
3.1	src/data_loader.py	5
3.2	src/model/model.py	5
3.3	src/model/loss.py	5
3.4	src/processing/post_processing/hough_detector.py	6
3.5	src/processing/post_processing/score_prediction.py	6
3.6	src/processing/training.py	6
3.7	src/processing/predict.py	7
3.8	src/processing/validate.py	7
3.9	src/save_validations.py	7
3.10	src/visualisation.py	7
3.11	run.py	8
3.12	apply.py	8
4	Running The Code	8
4.1	Training	8
4.1.1	Example Run: From Scratch	9
4.1.2	Example Run: Continue Training	15
4.2	Predicting	17
4.2.1	Example Run	17
4.3	Visualisations	20

1 Introduction

At its core, the code is an instance segmentation algorithm that takes in 3D image stacks and has 3 output classes: background, photoconverted-plus (PC+) vesicles and photoconverted-minus (PC-) vesicles. Details of the experiment that produces the data and its relevance can be found in this [preprint paper](#).

The vesicles that the code aims to segment are small scale (around 5 voxels wide) and approximately spherical objects. For this reason, the code is built around a UNet architecture, with a custom Hough-like post processing transformation that assumes perfectly spherical vesicle detections.

The code is written entirely using Python and makes heavy use of certain packages / software (which are detailed in the next section). As the code is meant to be open source, and in particular used by non-software experts, it is important that it is set up in an easy to use and configure fashion. For this reason, I have opted to go with a set-up that involves a simple to implement conda environment along with a collection of config files for tailoring the use of the code.

2 Prerequisites

In this section I will list the prerequisite python package knowledge in order to make sense of the rest of this document. I will provide basic details on each topic along with links to further documentation and information.

2.1 Zarr Data

The data for this code is saved and loaded in Zarr format, see [Zarr-Python documentation website](#) for more details. Zarr data can be thought of as a tailored version of Numpy data specifically for large N -dimensional data. How it does this, can be found via the Zarr-python website link above.

Zarr data can be equipped with custom key-value attributes, which allow you to store useful metadata. For this project the key attributes are:

- Axes: this is the orientation of the axes of the 3D image. We work with the convention of (z, y, x) .

- Offset: this is the offset of the data from the array (e.g. if the data has padding).
- Resolution: the resolution of the data, matching the axes order. Any unit can be used as long as it is consistent across the axes. We use the convention of angstrom ($\text{\AA}$), which is 0.1nm – this is used to ensure integer values for resolution.

2.2 Gunpowder

Gunpowder is a piece of software that allows us to build pipelines to analyse and train/predict on the data. The documentation can be found on the [Gunpowder website](#). Gunpowder is used heavily in the train and predict procedures in this code. I would advise working through the tutorial provided on the Gunpowder website to understand the software and then work over the train/predict pipelines.

For training, the key idea is that Gunpowder allows us to generate a subimage of specified size, at a random location within the full image, and to apply a collection of augmentations to this subimage. These augmentations are also applied to the ground-truth data, ensuring the labelling process is still correct. The training of the network is also built into the Gunpowder pipeline.

For prediction, the image is broken down into a grid of subimages, with the grid size determined by the subimage size used for training. The model then predicts on each grid square in turn (without any augmentations etc) and stitches them together to produce a prediction on the full image.

2.3 UNet

The model used for this project consists mainly of a UNet architecture, which was introduced in [this paper](#). I would highly recommend reading this paper in order to understand what a UNet is and how it works, bearing in mind the paper focuses on 2D image segmentation, whereas this project works with 3D image stacks.

Roughly speaking, a UNet is a collection of convolutions, downsampling, upsampling and skip-connections. The model is designed to not only be able to segment fine structure detail locally, but to also have some contextual

knowledge. This makes it the world leader for image segmentation of fine structure detail in large images, which is exactly what we are doing.

2.4 Hough transform

The Hough transform can be thought of as a technique to find imperfect instances of shapes within an image. In this project, we treat vesicles as perfectly spherical objects of a specified size. We make use of the Hough transform to find these spherical shapes within the probability maps the model returns.

2.5 Napari

Napari is a GUI software that allows for interactively viewing multi-dimensional python data. More information can be found on [their website](#). Note that Napari needs to be run using a terminal command, details of exactly how to do this can be found on the website. With this code, once the environment is correctly set up and activated using conda, the user can simply type 'napari' into terminal, and the Napari window will open. While a Napari window is open, the terminal window will not allow any further commands to be run within it. Once the window is closed, the terminal will then move onto a new command line and allow you to run new commands.

Tiff files are opened using File > Open File(s), while Zarr data should be loaded using File > Open Folder.

This project uses Napari to view the data along with any annotations. Napari allows you to annotate and label data, and so it is also used to help generate the ground-truth labels for training.

3 The Code

In this section I will list details on different files within the code, with a brief explanation of what they do and how to use them. I will try list these in an order that flows logically.

For now, I will assume the data has already been saved in the required format (e.g. Zarr data in the relevant directory structure).

3.1 src/data_loader.py

This file is used to load the Zarr data into memory, while performing checks. These checks include:

- Checking the directories are set up correctly.
- If loading training data, ensuring the attributes for the raw and ground truth data agree.

This file also allows for the creation of "target" data. This is simply a copy of the ground-truth data, but with data type altered to ensure it matches the requirements of the model (e.g. cross-entropy loss). If the input size of the model is larger than the ground truth data, the target data will also be padded with 0s to account for this.

Clahe versions of raw data can also be created with the code within this file.

3.2 src/model/model.py

This file contains the details of the model. It comprises of a UNet along with a convolutional head that results in a voxel-wise allocation of classes: (0:background, 1:PC+, 2:PC-). So we are left with 3 probability 'heat maps' for each of these classes. These need to be post-processed in order to extract the vesicles.

There is also some code for computing the shape of the output from the UNet. This can be used to ensure that the input size set in the config files is compatible with the UNet architecture. This check is done using the "check_output_shape.py" file.

3.3 src/model/loss.py

We use a customised version of the cross entropy loss. This is simply done to account for a masking of the data, i.e. if only part of the raw data has been annotated, we provide a mask to indicate which region of the data this is, and only data from within this masked region contributes to the loss.

3.4 src/processing/post_processing/hough_detector.py

This is where the custom built Hough-detector-like post processing algorithm is built. The idea is to convolve the probability maps that are the outputs of the model with a sphere of set size (the vesicle size).

The resulting convoluted probability maps are then used to find local peak maxima, representing the centres of potential vesicle candidates. Higher maxima values correspond to greater confidence of the model. These maxima values (along with their coordinates and label) are stored in decreasing order, and added one by one to an array corresponding to the final prediction.

Physical consistency checks are performed, including the exclusion of any candidate vesicle that intersects any already labelled vesicle.

3.5 src/processing/post_processing/score_prediction.py

This provides the method for which a prediction should be scored against provided hand labelled ground truth. The code computes recall, precision and f1-score for the PC+ and PC- classes separately, as well as computing their average.

3.6 src/processing/training.py

This is where the Gunpowder pipeline for training the model is built. Parameters relevant to training (that are not config parameters) are set here. For example, the weighting of the cross entropy loss (0.01, 1,1) for (background, PC+, PC-) is set in this file, to account for data imbalance. The optimiser (adam) is also introduced in this file.

As an important note, although most of the Gunpowder augmentations are set using the config files, the intensity augment is included in this file. This is done as the intensity augment requires you to provide the Gunpowder array as an argument (i.e. raw), but that is only defined within this file.

The "snapshot_every" argument was something that was inherited from a previous code. It is currently not set up right and needs to be looked into, should we want to keep it.

The statistics that should be stored during a training run are also defined in this file.

3.7 src/processing/predict.py

This is where the Gunpowder pipeline for predicting on an image is built. The pipeline has been set up to automatically account for the fact that the UNet's output has a smaller shape than the input: the size of image it predicts on is padded by exactly the difference in size to ensure the full image is predicted on.

3.8 src/processing/validate.py

This provides the process in which a validation is computed. The model being trained is used to predict on the validation data, post processed and then scored. Importantly, the loss used for validation is introduced itself in here, and so if it has been edited in the training.py file, it should also be updated here.

3.9 src/save_validations.py

This provides the method in which validation runs are saved. Namely, what data we want to save, and where we want to save it.

As it stands, for each validation we compute 9 scores (recall, precision and fscore for PC+,PC- and average). Validations are saved according to the validation runs that have the highest scores independently. So in total we get 9 saved validations (with potential duplicates for a run that is best in more than 1 score). The prediction itself is saved for each one of these too.

3.10 src/visualisation.py

This is where the method of loading the data (whether it is a validation run or a pure prediction) into Napari in order to visualise it is built. Checks for alignment of the raw data and the predicted labels are also done in this file.

This file can be called as part of the run.py or apply.py files (i.e. if the user says yes to visualising the result), but it can also be run manually to visualise previously generated data.

3.11 run.py

This is the code that should be run in order to train a model. This file makes use of a lot of the code explained above in order to build a training run. Understanding the previous files should make reading and understanding this file reasonably straight forward.

3.12 apply.py

This is the code that should be run in order to predict on some raw data, using a trained model. As with the run.py file, the understanding of this file should be straight forward if the other files are understood first.

4 Running The Code

4.1 Training

Running a training cycle is done using the run.py file. Before running the training run, the configurations should be set using the relevant config files, most importantly training_config.yaml: here you set the number of training iterations, how often to perform a validation run, and the input size for the model (along with a few other things).

The save_every value determines the number of iterations between manual saving of the model, regardless of performance. As it stands, these model checkpoints are just saved in the working directory and can then be moved afterwards.¹

Once the configs are complete, the user simply types "python run.py" into the terminal window and follows the on-screen instructions:

- Provide the path to the directory containing the Zarr data. This should be a Zarr group containing (at least) two other Zarr groups called "train" and "validate". Each of these should contain Zarr arrays called "raw" and "gt".
- Would you like to continue training a previous model? If yes, you have to provide the path to the previous model. This makes use of the way

¹This can be changed within the training.py file when calling the Gunpowder train node.

the model is saved via validation runs, and so the provided path should be to the saved validation run. It will then create a new validation save folder and copy over the information in the pretrained save. The model will then initialise itself based on the validation run that had the best performance under "best_score_name", defined in the config file.

- Would you like to visualise the prediction? If yes, this will display the validation run that performed the best under "best_score_name" using Napari, once training is complete.

While training is happening, the terminal window will display a progress bar. After every validation run, it will also print the current best validation scores, along with the corresponding iteration number.

4.1.1 Example Run: From Scratch

Below I shall include some print screens from an example training run. This is not an extensive run (i.e. more training iterations should improve the scores) but is simply an example of what to expect when the code is run. As can be seen by the print screen of the configs, this training run was for 10,000 iterations, with a validation done every 1,000 iterations. This results in a total of 10 validation runs, with all 9 scores calculated and printed to the screen after each validation. In this training run I have said 'no' to visualising the result. Below I will include an example of a training run that continues training on this model, and within that one I shall visualise the results and provide print screens.

A breakdown of the figures is as follows:

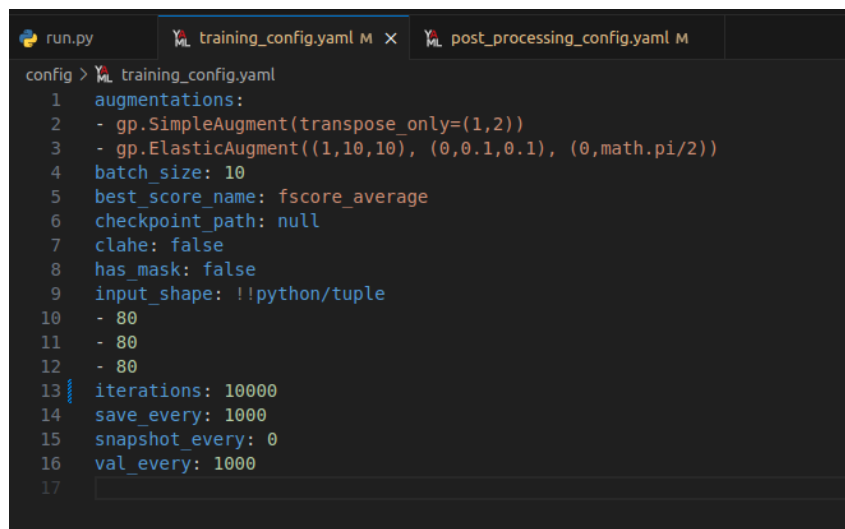
- Figures 1 to 3 show the training, post processing and model config files, respectively.
- Figure 4 shows the complete terminal results from the training run, including what I entered as the user. This training is done using the 19-13_training02.zarr data, trained from scratch and we do not visualise the data. Figures 5 and 6 then show the same terminal information, but in two print screens to help with readability.
- Figure 7 shows the files created once the training is complete. Within the 'saved_models' directory, it has created a directory with the date as its name, inside of which we have 2 directories (best_validations

and model_checkpoints) and 3 yaml files (best_scores, summary and training_config_used).

The best_validations directory contains 9 directories (one for each score). I have expanded the fscore_average one as an example of the content: we have the post-processed prediction saved as zarr data, along with 2 csv files, the candidate vesicles and the stats of the validation run – the former contains the centre location, score and label of every accepted vesicle candidate, and the latter contains the training iteration number used for the validation, all 9 scores for the validation run and the loss value of the validation.

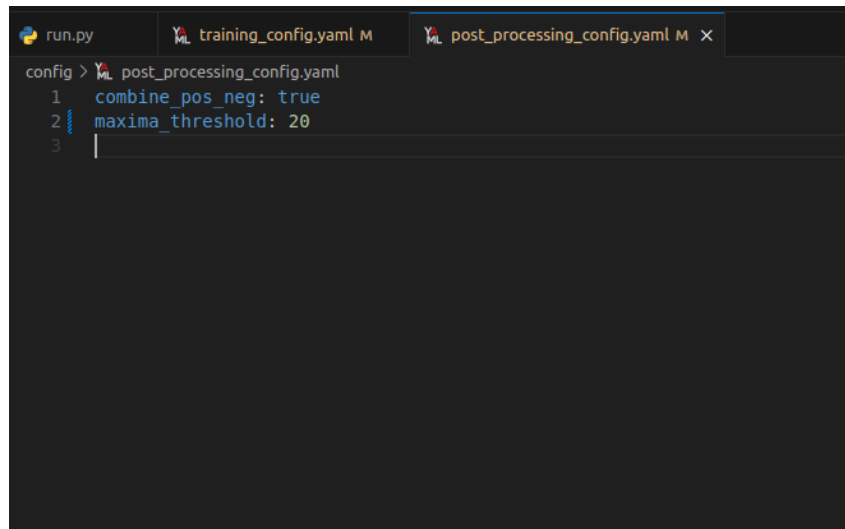
The model_checkpoints directory contains 9 files corresponding to the model checkpoints that provided the best result for each of the 9 scores.

Additionally to the saved_models directory, we also have 10 model checkpoints saved in the working directory. These correspond to the save_every config parameter. These are simply the model checkpoints for the training iteration of the corresponding tag (e.g. model_checkpoint_3000 corresponds to the model's checkpoint on training iteration 3000, regardless of performance). These model checkpoints can then be moved to another directory (e.g. the Model_checkpoints directory) once training is complete.



```
config > training_config.yaml
1 augmentations:
2   - gp.SimpleAugment(transpose_only=(1,2))
3   - gp.ElasticAugment((1,10,10), (0,0.1,0.1), (0,math.pi/2))
4 batch_size: 10
5 best_score_name: fscore_average
6 checkpoint_path: null
7 clahe: false
8 has_mask: false
9 input_shape: !!python/tuple
10  - 80
11  - 80
12  - 80
13 iterations: 10000
14 save_every: 1000
15 snapshot_every: 0
16 val_every: 1000
17
```

Figure 1: Training configs.



The screenshot shows a code editor with three tabs: 'run.py', 'training_config.yaml M', and 'post_processing_config.yaml M X'. The 'post_processing_config.yaml' tab is active, displaying the following YAML configuration:

```
config > post_processing_config.yaml
1 combine_pos_neg: true
2 maxima_threshold: 20
3
```

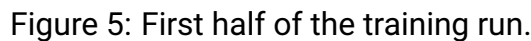
Figure 2: Post processing configs.



The screenshot shows a code editor with four tabs: 'run.py', 'training_config.yaml M', 'post_processing_config.yaml M', and 'model_config.yaml X'. The 'model_config.yaml' tab is active, displaying the following YAML configuration:

```
config > model_config.yaml
1 constant_upsample: true
2 downsample_factors:
3   - !!python/tuple
4     - 2
5     - 2
6     - 2
7   - !!python/tuple
8     - 2
9     - 2
10    - 2
11 fmap_inc_factors: 5
12 fmaps: 32
13 padding: valid
14
```

Figure 3: Model configs.



```
Terminal
50% [redacted] | 5000/10000 [6:31:47<6:38:30, 4.78s/it]
Running validation...
100% [redacted] | 32/32 [00:00<00:00, 44.61it/s]
('precision_1': (0.5833333333333334, 'Iteration 1000'), 'recall_1': (0.6451612903225806, 'Iteration 4000'), 'fscore_1': (0.5607476635514019, 'Iteration 4000'), 'precision_2': (0.17037037037037037, 'Iteration 5000'), 'recall_2': (0.4262295081967213, 'Iteration 4000'), 'fscore_2': (0.2346938775510204, 'Iteration 5000'), 'precision_average': (0.35838106147384496, 'Iteration 5000'), 'recall_average': (0.5356953992596509, 'Iteration 4000'), 'fscore_average': (0.3962943071965628, 'Iteration 5000'))
Resuming training...
60% [redacted] | 6000/10000 [7:50:10<5:19:23, 4.79s/it]
Running validation...
100% [redacted] | 32/32 [00:00<00:00, 49.29it/s]
('precision_1': (0.5979301443298969, 'Iteration 6000'), 'recall_1': (0.6451612903225806, 'Iteration 4000'), 'fscore_1': (0.6105263157894737, 'Iteration 6000'), 'precision_2': (0.17037037037037037, 'Iteration 5000'), 'recall_2': (0.45901639344262296, 'Iteration 6000'), 'fscore_2': (0.24034334763948498, 'Iteration 6000'), 'precision_average': (0.3803644210021577, 'Iteration 6000'), 'recall_average': (0.5413361537105588, 'Iteration 6000'), 'fscore_average': (0.4254348317144793, 'Iteration 6000'))
Resuming training...
70% [redacted] | 7000/10000 [9:08:34<3:58:54, 4.78s/it]
Running validation...
100% [redacted] | 32/32 [00:00<00:00, 49.99it/s]
('precision_1': (0.6309523809523809, 'Iteration 7000'), 'recall_1': (0.6451612903225806, 'Iteration 4000'), 'fscore_1': (0.6105263157894737, 'Iteration 6000'), 'precision_2': (0.29009767441860467, 'Iteration 7000'), 'recall_2': (0.45901639344262296, 'Iteration 6000'), 'fscore_2': (0.3401360544217687, 'Iteration 7000'), 'precision_average': (0.4608250276854928, 'Iteration 7000'), 'recall_average': (0.5413361537105588, 'Iteration 6000'), 'fscore_average': (0.4695030554594719, 'Iteration 7000'))
Resuming training...
80% [redacted] | 8000/10000 [10:26:57<2:39:13, 4.78s/it]
Running validation...
100% [redacted] | 32/32 [00:00<00:00, 50.50it/s]
('precision_1': (0.6309523809523809, 'Iteration 7000'), 'recall_1': (0.6451612903225806, 'Iteration 4000'), 'fscore_1': (0.6105263157894737, 'Iteration 6000'), 'precision_2': (0.29009767441860467, 'Iteration 7000'), 'recall_2': (0.45901639344262296, 'Iteration 6000'), 'fscore_2': (0.3401360544217687, 'Iteration 7000'), 'precision_average': (0.4608250276854928, 'Iteration 7000'), 'recall_average': (0.5413361537105588, 'Iteration 6000'), 'fscore_average': (0.4695030554594719, 'Iteration 7000'))
Resuming training...
90% [redacted] | 9000/10000 [11:45:18<1:19:47, 4.79s/it]
Running validation...
100% [redacted] | 32/32 [00:00<00:00, 48.78it/s]
('precision_1': (0.6309523809523809, 'Iteration 7000'), 'recall_1': (0.6666666666666666, 'Iteration 9000'), 'fscore_1': (0.6424870466321244, 'Iteration 9000'), 'precision_2': (0.34210526315789475, 'Iteration 9000'), 'recall_2': (0.45901639344262296, 'Iteration 6000'), 'fscore_2': (0.3795620437956204, 'Iteration 9000'), 'precision_average': (0.4810526315789474, 'Iteration 9000'), 'recall_average': (0.5464480874316939, 'Iteration 9000'), 'fscore_average': (0.5110245452138724, 'Iteration 9000'))
Resuming training...
100% [redacted] | 10000/10000 [13:03:42<00:00, 4.70s/it]
Running final validation...
100% [redacted] | 32/32 [00:00<00:00, 48.73it/s]
('precision_1': (0.631578947368421, 'Iteration 10000'), 'recall_1': (0.6666666666666666, 'Iteration 9000'), 'fscore_1': (0.6424870466321244, 'Iteration 9000'), 'precision_2': (0.34210526315789475, 'Iteration 9000'), 'recall_2': (0.45901639344262296, 'Iteration 6000'), 'fscore_2': (0.3795620437956204, 'Iteration 9000'), 'precision_average': (0.4810526315789474, 'Iteration 9000'), 'recall_average': (0.5464480874316939, 'Iteration 9000'), 'fscore_average': (0.5110245452138724, 'Iteration 9000'))
04/02 10:18 nrd635 fms-302740 ~/Documents/VesicleDetection
(VesicleDetectionEnv) ➜
```

Figure 6: Second half of the training run.

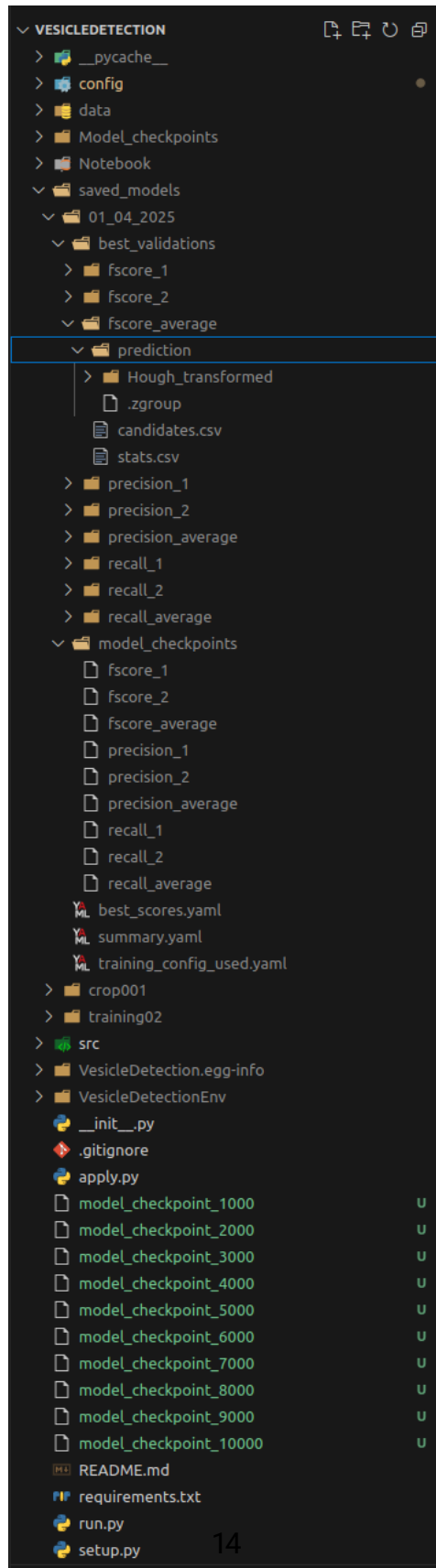


Figure 7: Results of created files from training run.

4.1.2 Example Run: Continue Training

Below is an example of continuing to train the model trained above. Here I have chosen to keep the training data the same, but this is not strictly necessary. For the sake of time, I have only run 1000 iterations with validations happening every 500 iterations (i.e. 2 validation runs). Again figures explained below.

- Figure 8 shows the terminal for the continue training run. When prompted as to which model to use, the user has to enter the directory within the saved_models folder (i.e. the one that contains the best_validations etc). This will then copy and paste all the saved data from that directory into a new one and resume training, using the model checkpoint that corresponds to the best_score_name defined in the config files. When displaying the scores upon a validation run, an asterisk is used to indicate the iteration number corresponding to a previous run while no asterisk means it is from the current training run. For example the best fscore_1 has Iteration 9000*, which means it corresponds to the 9000th iteration from the first run of 10,000 above. On the other hand, fscore_2 has Iteration 1000 which means the 1000th iteration of the new run (i.e. the 11,000 total iteration).
- Figure 9 shows the Napari output when visualising a training run. It displays the prediction that had the best score under best_score_name. The target (i.e. hand drawn ground truth vesicles) is also displayed. The programme loads both Hough transformed and Target as image layers, but these can be turned into Label layers by right clicking on them. When the Napari window opens, the target labels are displayed using pink (PC+) and white (PC-) colours, while the predicted labels are all shown in green. This is done to provide clear contrast between the two. The differentiation of the predicted PC+ vs PC- labels can be seen by either changing the 'colourmap' of Hough Transformed, or by converting it to a labels layer.


```
Terminal
04/02 11:04 nrd635 fms-302740 ~/Documents/VesticleDetection
(VesticleDetectionEnv) → python run.py
Provide path to zarr container: data/19-13_training02.zarr
-----
Would you like to continue training a previous model? (y/n): y
Provide path to the saved model: saved_models/01_04_2025
-----
Would you like to visualise the prediction? (y/n): y
-----
Loading data from data/19-13_training02.zarr...
Resuming training from model with previous scores:
{'fscore_1': (0.6424870466321244, 'Iteration 9000*'), 'fscore_2': (0.3795620437956204, 'Iteration 9000*'), 'fscore_average': (0.5110245452138724, 'Iteration 9000*'),
'precision_1': (0.631578947368421, 'Iteration 10000*'), 'precision_2': (0.34210526315789475, 'Iteration 9000*'), 'precision_average': (0.4810526315789474, 'Iteration
9000*'), 'recall_1': (0.6666666666666666, 'Iteration 9000*'), 'recall_2': (0.45901639344262296, 'Iteration 6000*'), 'recall_average': (0.5464480874310939, 'Iterati
on 9000*')}
Starting training for 1000 iterations...
50% | 500/1000 [39:07<39:10, 4.70s/it]
Running validation...
100% | 32/32 [00:00<00:00, 49.54it/s]
{'fscore_1': (0.6424870466321244, 'Iteration 9000*'), 'fscore_2': (0.3850931677018634, 'Iteration 500'), 'fscore_average': (0.5110245452138724, 'Iteration 9000*'),
'precision_1': (0.6707317073170732, 'Iteration 500'), 'precision_2': (0.34210526315789475, 'Iteration 9000*'), 'precision_average': (0.4903658536585366, 'Iteration 50
0'), 'recall_1': (0.6666666666666666, 'Iteration 9000*'), 'recall_2': (0.5081967213114754, 'Iteration 500'), 'recall_average': (0.5497972853869205, 'Iteration 500')}
Resuming training...
100% | 1000/1000 [1:18:18<00:00, 4.70s/it]
Running final validation...
100% | 32/32 [00:00<00:00, 51.19it/s]
{'fscore_1': (0.6424870466321244, 'Iteration 9000*'), 'fscore_2': (0.4109589041095891, 'Iteration 1000'), 'fscore_average': (0.5224193427651771, 'Iteration 1000'),
'precision_1': (0.6707317073170732, 'Iteration 500'), 'precision_2': (0.35294117647058826, 'Iteration 1000'), 'precision_average': (0.49809281045751034, 'Iteration 10
00'), 'recall_1': (0.6666666666666666, 'Iteration 9000*'), 'recall_2': (0.5081967213114754, 'Iteration 500'), 'recall_average': (0.5577295963335096, 'Iteration 1000')}
})
]
```

Figure 8: Continuing training. An asterisk after an iteration number indicates the iteration number from the previous training run.

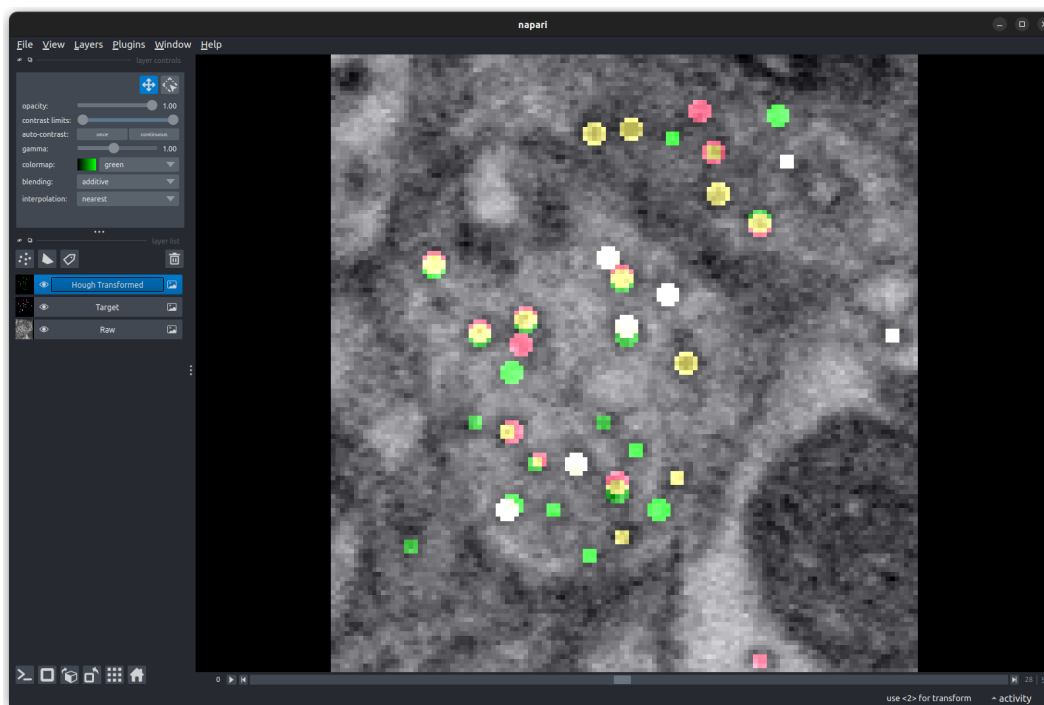


Figure 9: Napari window from visualising the best validation from the continue training run.

4.2 Predicting

Running a pure prediction is done using the "apply.py" file. The main configuration that is important here is the `input_shape` within the training config file. This **MUST** match the `input_shape` used for whichever model is to be used. As it stands there is no direct check for this matching criteria, but it would be a good idea to add this and prompt the user to update the config file and run the `apply.py` file again.

When the file is run, the user simply follows the on screen instructions again.

- Provide the path to the directory containing the Zarr data. This should be a Zarr group containing (at least) the Zarr group "predict". Predict should contain a Zarr array called "raw".
- Provide the model checkpoint. Here the user needs to provide the path directly to the model checkpoint itself. So it is not enough to simply enter the collection of saved validations, but it should point specifically to ".../fscore_average", for example. This could be updated to be similar to the continue training prompt from `run.py`: i.e. the user simply provides the saved validations, and then the config file is used to determine which checkpoint to use.
- Would you like to visualise the prediction? If yes, this will display the prediction using Napari.

When the prediction is complete, the model will also print the number of PC+ and PC- vesicles it detected within the terminal window. This is not saved anywhere, and so should be noted should the user want it.

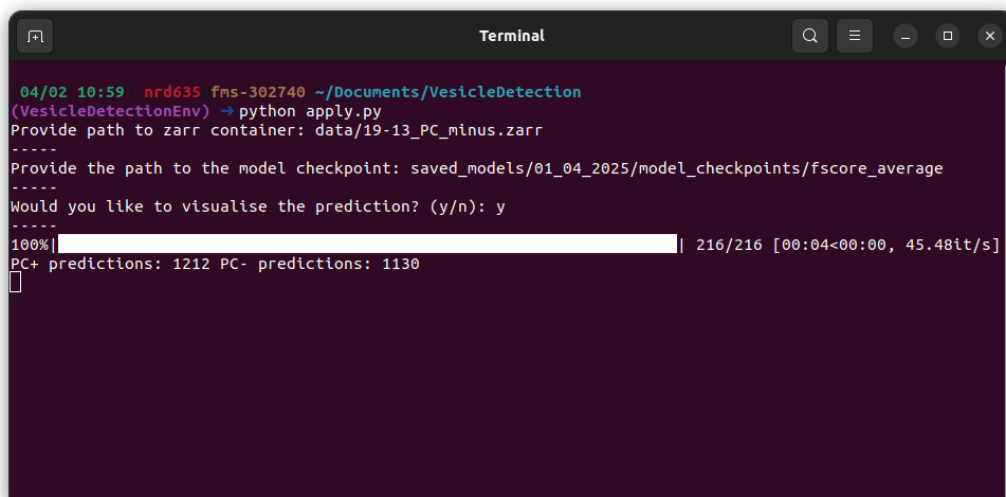
4.2.1 Example Run

Again, a summary of the figures is below:

- Figure 10 shows the terminal output when running a prediction run on the 19-13_PC_minus.zarr data. Here we have used the model trained above (i.e. `saved_models/01_04_2025`), specifically the model that performed best on `fscore_average`. We have then asked to visualise the result, which is shown in Figure 11. Note that the terminal result displays the number of PC+ and PC- predictions. Additionally note that the terminal command has not completed (i.e. you can't enter a new

command), this is because the Napari window is open. Once you close the Napari window, it will complete and you can enter a new command.

- Figure 11 shows the Napari window containing the prediction. Note, as it stands, the labels are loaded in as an image layer in Napari. This is to avoid accidentally erasing a label once loaded in. However, should the user want to see the labels as a label layer, you simply right click on the Hough Transformed layer and convert it to a label layer.
- Figure 12 shows how the prediction is saved: within the directory containing the prediction data, a directory named Predictions is created, and in that a Zarr group (whose name is the date of prediction) is created and the post processed prediction is saved as Zarr data inside Hough_transformed.



```
04/02 10:59 nrd635 fms-302740 ~/Documents/VesicleDetection
(VesicleDetectionEnv) → python apply.py
Provide path to zarr container: data/19-13_PC_minus.zarr
-----
Provide the path to the model checkpoint: saved_models/01_04_2025/model_checkpoints/fscore_average
-----
Would you like to visualise the prediction? (y/n): y
-----
100%|████████████████████████████████████████████████████████████████████████████████| 216/216 [00:04<00:00, 45.48it/s]
PC+ predictions: 1212 PC- predictions: 1130
█
```

Figure 10: Running a prediction.

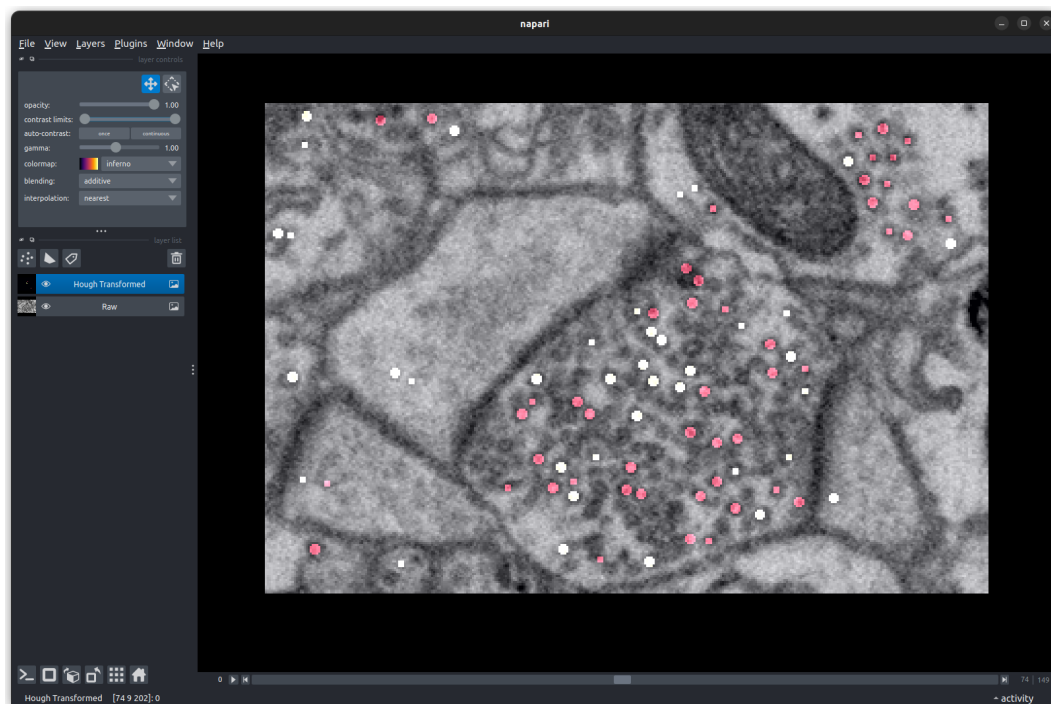


Figure 11: Output from visualising the above prediction run.

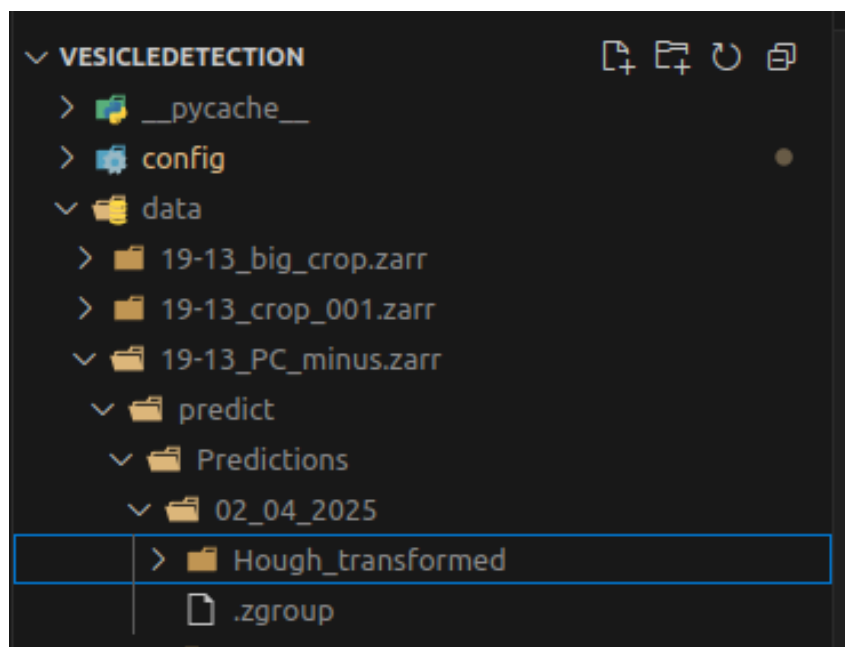


Figure 12: Saving of a prediction.

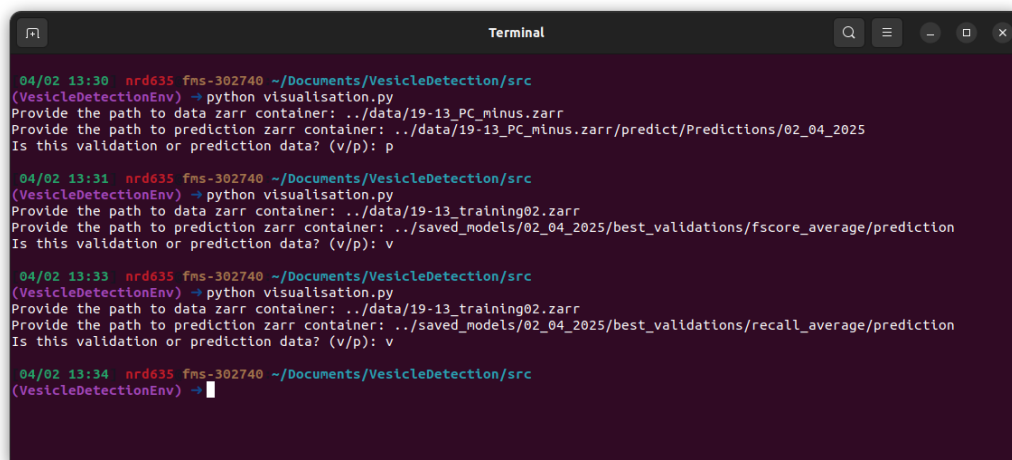
4.3 Visualisations

Here I will provide examples of running the visualisation.py code. The print screen in Figure 13 shows 3 different runs:

1. A visualisation of the prediction done above.
2. A visualisation of the validation which had best fscore_average
3. A visualisation of the validation which had best recall_average

I have not included print screens of the Napri windows, as they are equivalent, respectively, to Figure 11 and Figure 9, and similarly for the recall_average visualisation.

Note that the directory of the terminal is within the src folder (as this is where the visualisation file is contained). This is why each provided path starts with '../'. The code could as equally be run from the main VesicleDetection directory with 'python run src/visualisation.py', and then the paths would not require the '../' at the beginning.



```
04/02 13:30 nrd635 fms-302740 ~/Documents/VesicleDetection/src
(VesicleDetectionEnv) → python visualisation.py
Provide the path to data zarr container: ../data/19-13_PC_minus.zarr
Provide the path to prediction zarr container: ../data/19-13_PC_minus.zarr/predict/Predictions/02_04_2025
Is this validation or prediction data? (v/p): p

04/02 13:31 nrd635 fms-302740 ~/Documents/VesicleDetection/src
(VesicleDetectionEnv) → python visualisation.py
Provide the path to data zarr container: ../data/19-13_training02.zarr
Provide the path to prediction zarr container: ../saved_models/02_04_2025/best_validations/fscore_average/prediction
Is this validation or prediction data? (v/p): v

04/02 13:33 nrd635 fms-302740 ~/Documents/VesicleDetection/src
(VesicleDetectionEnv) → python visualisation.py
Provide the path to data zarr container: ../data/19-13_training02.zarr
Provide the path to prediction zarr container: ../saved_models/02_04_2025/best_validations/recall_average/prediction
Is this validation or prediction data? (v/p): v

04/02 13:34 nrd635 fms-302740 ~/Documents/VesicleDetection/src
(VesicleDetectionEnv) →
```

Figure 13: Examples of running the visualisation code. The first run corresponds to the prediction done above. The second run is the validation from the continue training that had the best fscore_average, and the third is the same but for recall_average.